Projeto TaskTracker

Fase 1: Entrega Inicial: Planejamento Lógico e Arquitetural

Bootcamp II · Engenharia de Software e Versionamento de Código

Estudante: Luiz Felipe Rocha · RA: 22604023 · Curso: Desenvolvimento de Software

1. Contexto e problema

Pessoas que acumulam estudos, trabalho e compromissos pessoais costumam perder prazos porque registram tarefas de forma dispersa (anotações soltas, mensagens, memória). O TaskTracker propõe uma ferramenta simples, executada no terminal, que centraliza o cadastro das tarefas e permite consultá-las a qualquer momento, com prioridade e data limite claramente identificadas.

Objetivo da solução: permitir que o usuário cadastre tarefas com dados mínimos garantidos (título válido e prioridade padronizada) e visualize a lista completa de forma organizada.

2. Decomposição do problema

O problema foi dividido em partes menores e independentes, cada uma tratada por uma função específica no código:

Subproblema	Responsabilidade	Função planejada
Navegação	Exibir o menu e manter o programa ativo até o usuário sair	exibir_menu() e main()
Entrada do título	Ler o título e rejeitar valores vazios ou só com espaços	ler_titulo()
Entrada da prioridade	Aceitar apenas Alta, Média/Media ou Baixa	ler_prioridade()
Cadastro	Montar a tarefa, definir status Pendente e guardar na lista	cadastrar_tarefa()
Listagem	Informar lista vazia ou exibir todas as tarefas formatadas	visualizar_tarefas()

3. Arquitetura de dados

Cada tarefa é representada como um **dicionário** Python. Todas as tarefas ficam armazenadas em uma **lista** em memória durante a execução do programa.

Campo	Tipo	Obrigatório	Regra de validação
titulo	String	Sim	Não pode ser vazio nem conter apenas espaços. Repetir a pergunta até ser válido.
descricao	String	Não	Texto livre.

prioridade	String	Sim	Somente "Alta", "Média" (ou "Media") e "Baixa". Fora disso: mensagem de erro e nova pergunta.
data_limite	String	Não	Data planejada para conclusão (formato livre ou DD/MM/AAAA).
status	String	Automático	Definido como "Pendente" no momento do cadastro.

Exemplo de uma tarefa armazenada:

```
tarefas = [
    {
        "titulo": "Estudar Git",
        "descricao": "Revisar comandos de commit e push",
        "prioridade": "Alta",
        "data_limite": "20/10/2026",
        "status": "Pendente"
    }
]
```

4. Algoritmo em português (pseudocódigo)

```
ALGORITMO TaskTracker
INÍCIO
  CRIAR lista vazia chamada tarefas

  REPITA
    EXIBIR "1. Cadastrar nova tarefa"
    EXIBIR "2. Visualizar tarefas cadastradas"
    EXIBIR "3. Sair da aplicação"
    LER opcao

    SE opcao = 1 ENTÃO
      REPITA
        LER titulo
        REMOVER espaços do início e do fim de titulo
        SE titulo estiver vazio ENTÃO
          EXIBIR "Erro: o título é obrigatório"
        FIM SE
      ATÉ QUE titulo não esteja vazio

      LER descricao

      REPITA
        LER prioridade
        SE prioridade NÃO for "Alta", "Média", "Media" ou "Baixa" ENTÃO
          EXIBIR "Erro: prioridade inválida"
        FIM SE
      ATÉ QUE prioridade seja válida

      LER data_limite
      status <- "Pendente"
      CRIAR tarefa com titulo, descricao, prioridade,
            data_limite e status
      ADICIONAR tarefa em tarefas
      EXIBIR "Tarefa cadastrada com sucesso"

    SENÃO SE opcao = 2 ENTÃO
      SE tarefas estiver vazia ENTÃO
        EXIBIR "Nenhuma tarefa cadastrada no momento"
      SENÃO
        PARA CADA tarefa EM tarefas FAÇA
          EXIBIR titulo, descricao, prioridade,
                 data_limite e status
        FIM PARA
      FIM SE

    SENÃO SE opcao = 3 ENTÃO
      EXIBIR "Encerrando o TaskTracker"

    SENÃO
      EXIBIR "Opção inválida"
    FIM SE
  ATÉ QUE opcao = 3
FIM
```

5. Fluxo do menu

- O menu é exibido em um laço contínuo (**while**) até a escolha da opção 3.
- Opção 1 chama o cadastro, que só termina com título e prioridade válidos.
- Opção 2 chama a listagem, que trata o caso de lista vazia.
- Qualquer outra entrada mostra "Opção inválida" e o menu reaparece.

6. Mapeamento para Git e GitHub

Estrutura de pastas planejada para o repositório local e remoto:

```
tasktracker-projeto/
├── README.md                    # Documentação e instruções de uso
├── .gitignore                   # Ignora __pycache__/, *.pyc e .venv/
├── docs/
│   └── planejamento_logico.pdf  # Este documento (Fase 1)
└── src/
    └── main.py                  # Código-fonte da aplicação
```

Plano de versionamento (histórico de commits):

Etapa	Comando	Conteúdo
1. Inicialização	git init	Criação do repositório e da estrutura de pastas.
2. Commit inicial	git commit	README.md, .gitignore e docs/planejamento_logico.pdf.
3. Desenvolvimento	git commit	src/main.py com menu e loop principal.
4. Regras e validações	git commit	Validação de título não vazio e de prioridades.
5. Publicação	git push -u origin main	Vinculação e envio ao repositório remoto no GitHub.

7. Tecnologias previstas

- **Python 3**: linguagem da aplicação, sem bibliotecas externas.
- **Git**: controle de versão local com commits por etapa.
- **GitHub**: hospedagem pública do repositório (branch main).
- **Docker**: previsto para a Fase 3 (empacotamento e deploy).